Comparing Red–Black and AVL Trees

CSD2183 Data Structures — Project 1 — Assignment Brief

Your team will implement red–black and AVL trees in C++, test them, and compare their performance experimentally, using as a baseline `std::map`, which the standard libraries of GCC and Clang implement as a red–black tree. You will also analyze a published study that compares the two trees and relate its findings to your own results. Your team must use generative AI within the rules in Section 7. How critically you use it counts, not how much. Assessment emphasizes the quality of your investigation, your interpretation of the evidence, and your critical evaluation of AI assistance.

1 Background

1.1 Red–Black versus AVL Trees

Both trees restore balance after insertion or deletion by rotating subtrees, but they differ in how strictly they enforce it. In an AVL tree, the balance factor of every node, defined as the height of its right subtree minus the height of its left subtree, must be -1, 0, or 1, which limits the height of a tree with n nodes to about $1.44 \lg n$. The color rules of a red–black tree allow a height of up to about $2 \lg n$. Figure 1 shows 11 keys for which a red–black tree can reach depth 4 but no AVL tree can.

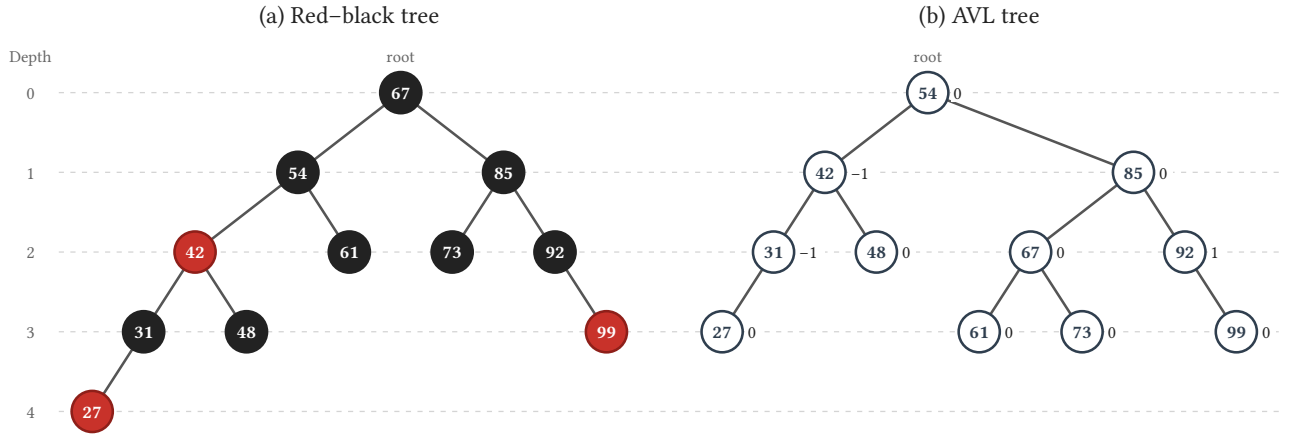


Figure 1: A red–black tree (a) and an AVL tree (b) holding the same 11 keys. The number beside an AVL node is its balance factor.

Tighter balance may shorten searches, whereas looser balance may make updates cheaper. Your experiments must determine whether either effect makes one tree faster on a given workload.

Chapter 13 of Cormen *et al.* (2022) specifies the red–black tree, and its Problem 13-3 introduces the AVL tree but develops only insertion. Brown (2025) compares the AVL tree with three variants of the red–black tree. In Brown's terms, the red–black tree of Cormen et al. is bottom-up, because it restores balance on the way back up from an inserted or deleted node. Brown calls deletion customary if a deleted node that has two children is always replaced by its successor, or always by its predecessor; Cormen et al. use the successor. Implement the red–black tree as Cormen et al. do, with bottom-up rebalancing and customary deletion. For AVL deletion, implement either the customary algorithm or the alternative one described in Section 1.2 of Brown (2025), which replaces a deleted node that has two children by its predecessor or its successor, whichever lies in the taller subtree, or either if the two subtrees are equally tall. The choice is yours, but your report must state which one you implemented and show that it preserves the AVL property and runs in $O(\lg n)$ time.

1.2 Intended Learning Outcomes

On completing the project, you should be able to:

- **Implement and test dynamic data structures:** Implement search, insertion, and deletion in red–black and AVL trees, maintain their invariants, manage memory correctly, and design tests that expose defects.
- **Evaluate algorithmic trade-offs:** Weigh the relative advantages of different self-balancing methods by analyzing their empirical running time and tree depth.
- **Analyze scientific literature:** Critically analyze a research article, relating its claims to your own results.
- **Critically assess generative AI:** Incorporate chat-based AI into your workflow, evaluate its suggestions, and justify decisions to adopt, amend, or reject them.

2 Program Specification

The starter code (Section 3) contains the complete program except for the two trees, which your team implements.

2.1 Commands

The program runs a command file on one of three structures:

```
./tree <rb|avl|map> <command_file>
```

The selectors `rb` and `avl` run your trees, and `map` runs the `std::map` baseline. Each line of a command file holds one command, with fields separated by single spaces; empty lines and lines beginning with `#` are ignored. Keys and values are signed 64-bit integers (`std::int64_t`), and each run starts with an empty structure.

Command	Effect	Output
`I <key> <value>`	Inserts the pair; if the key is already present, keeps its existing value	None
`D <key>`	Deletes the key if it is present	None
`S <key>`	Searches for the key	`S <key> found <value>` or `S <key> absent`
`P`	Prints all keys	`P` followed by the keys in ascending order
`X`	Exports the shape of the tree	See Section 2.2

Command files used for grading contain up to 10^6 commands; `P` and `X` appear only in small files.

2.2 Shape Export

The member function `export_shape` of each tree, which you implement, writes the output of `X`: the line `root <key>`, or `root -` for an empty tree, followed by one line per node in ascending key order. Each node line reads `<key> <attribute> <left> <right>`, where `<attribute>` is `R` or `B` for the red–black tree and the balance factor `-1`, `0`, or `1` for the AVL tree, and `<left>` and `<right>` are the keys of the children, or `-` if a child is absent. Export the tree as it is, without rebuilding or rebalancing it; omit NIL leaves, including the sentinel of Cormen et al. if you use one, and end every line with a newline. Write nothing else to standard output, and send any diagnostics to standard error. With `map`, the program prints `X unsupported` instead, because `std::map` gives no access to its tree.

Correct implementations may build different trees, so `X` output is not compared byte for byte. Instead, it must describe a binary search tree that holds the expected keys and satisfies the invariants of its type:

- **Red–black tree:** The root of a nonempty tree is black, no red node has a red child, and every path from the root to a NIL leaf contains the same number of black nodes.

- **AVL tree:** Every balance factor is `-1`, `0`, or `1` and agrees with the subtree heights.

For example, given the command file

```
# data/example.txt
# The expected output of ./tree rb, ./tree avl, and ./tree map with this file
# is in example_rb.expected, example_avl.expected, and example_map.expected.
# The X blocks for rb and avl show one valid tree each; yours may differ.
I 8 100
I 3 200
I 17 300
I 3 999
S 3
S 10
D 8
S 8
P
X
```

the invocation `./tree rb data/example.txt` may write:

```
S 3 found 200
S 10 absent
S 8 absent
P 3 17
root 17
3 R - -
17 B 3 -
```

The first four lines are the same for every correct implementation; the last three show one valid red–black tree.

2.3 Implementation Rules

Implement node storage, rotations, insertion, search, deletion, and rebalancing yourselves. Do not use standard-library containers (except `std::vector` and `std::array`) or third-party data-structure code; `std::map` may appear only in the baseline. You may learn the algorithms, including AVL deletion, from any source, such as textbooks or websites, if you cite it, but write the code yourselves. Release every node when it is deleted or when the tree is destroyed.

The command `make` builds the program `tree`, and `make debug` builds `tree_debug`, a version with sanitizers that detect memory errors and undefined behavior. On a single reference machine, `tree` must process each command file of the instructor tests within one minute, or marks are deducted (Section 9); apart from this limit, absolute running times do not affect grades. Gradescope shows how long `tree_debug`, which is much slower than `tree`, takes on its two files with 10^6 commands; if both finish within one minute there, `tree` meets the limit.

3 Starter Code and Repository

Set up your team repository as follows:

1. Download `self_balancing_trees.zip` from xSITe.
2. One team member creates an empty private GitHub repository named `csd2183-project1-team-<n>`, where `<n>` is your project team number on xSITe, adds the other team members and all instructors as collaborators (`mgastner`, `drbingjiexu`, `chiewtk`, `AFKeeWoonTan`, and `stefanroata`), and clones the repository.

3. The same team member extracts the ZIP file into the cloned folder, checks that the `Makefile`, the hidden `.gitignore` file, and the hidden `.githooks` folder are at the top level rather than in a subfolder, runs the following commands in that folder, and pushes:

```
git config core.hooksPath .githooks
git add --chmod=+x .githooks/commit-msg
git add .
git commit -m "Add starter code" -m "AI: none"
```

4. The other team members clone the repository and run `git config core.hooksPath .githooks` in their clones.

The starter code contains the following files and directories:

Path	Contents	Editable
`Makefile`	Build rules for `make`, `make debug`, and `make bench`	No
`include/`	Headers declaring the trees and the baseline	No
`src/main.cpp`	The program, except for the trees	No
`src/` (other files)	Stubs for both trees, and any files your team adds	Yes
`bench/`	An example benchmark	Yes
`bench/nanobench/`	The nanobench benchmarking library	No
`data/`	Example command files with their expected output	No
`tests/`	Your team's tests, initially one example	Yes
`ai_logs/`	Your AI interaction logs	Yes
`report/`	The Microsoft Word template for your report	Yes
`.githooks/`	A hook that checks commit messages	No

Do not change the files marked "No"; Gradescope restores the original `Makefile`, `include/`, `src/main.cpp`, and `bench/nanobench/` before building your code. The comments in the headers specify what each member function must do, and the instructor tests call every member function directly. Define your node types, such as `struct RBNode`, in `src/`. Because the classes are fixed, write helper functions as free functions in your source files.

Gradescope builds your code with `g++` 15.2 on Linux, and that build is authoritative. On Windows, run the programs with a leading `.\`, for example `.\tree.exe`; if `make debug` fails there, rely on the debug build on Gradescope.

The assessed commit is the latest commit on the default branch pushed before your team's final upload to xSITe, and your final Gradescope submission must match it. Commit regularly, in small steps with meaningful messages. Never force-push or otherwise rewrite pushed history; doing so may be treated as academic misconduct. You may use branches and pull requests, but merge them with merge commits rather than squash merges, so that every commit stays in the history of the default branch. End every commit message, apart from merge commits, with a line that declares the origin of its changes, either `AI: none` or the names of the AI log files that influenced its changes (Section 7.3), separated by a comma and a space, for example:

```
AI: 2026-10-02_chatgpt_rotation-bug.md, 2026-10-04_claude_delete-cases.md
```

Once set up, Git rejects commits without this line; commits made on the GitHub website need it too, although nothing checks them there. If a commit without this line, or with a wrong one, reaches the repository anyway,

do not rewrite it; declare its origin before your final upload to xSITe in an empty commit that names it by its hash, for example:

```
git commit --allow-empty -m "Declare origin of a1b2c3d" -m "AI: none"
```

Changes whose origin is not declared in this way are treated as undeclared AI use.

4 Testing and Experiments

4.1 Testing Your Code

A correct tree produces the same `S` and `P` output as the `map` selector, so use `map` as the reference for every command file, and check `X` output against the rules in Section 2.2. Run your tests with `tree_debug`, and treat every error report from the sanitizers, such as `ERROR: AddressSanitizer: heap-use-after-free`, as a defect, even when the output is correct.

Gradescope runs the instructor tests on every submission and shows you the results, so submit whenever you want feedback; a test fails on any sanitizer report, even when the output is correct.

4.2 Your Team's Tests

Submit ten test files of your own in `tests/`, besides `tests/example.txt`, each a command file with the extension `.txt` and at most 10,000 commands. Begin each file with a comment that states the behavior or defect it targets, as in `tests/example.txt`. Every file must run with `./tree map` without error, and no test may depend on one particular valid `X` output.

After the deadline, each team's tests run on every other team's `tree_debug`, once with each tree, with at most 5 seconds and 64 MB of output per run. Your code earns credit for passing other teams' tests, and your tests earn credit for exposing defects in other teams' code. Tests that break these rules or target no plausible defect reduce your score.

4.3 Experiments

Measure running times with nanobench: `make bench` builds each `.cpp` file in `bench/` into a program, and `bench/example.cpp` shows how to time searches, insertions, and deletions. For other compiler options, such as those of the article, build your benchmarks with your own commands and keep them in the repository. Compare the three structures at four or more sizes, including 2^{18} keys, on the same machine and with the same keys, for insertion and deletion, each in sorted and in random order, and for search after sorted and after random insertion. For both trees, also measure the height, which is the largest depth of any node, and the average depth of the nodes, with the member functions `height` and `average_depth`. Measure both at the sizes of your insertion benchmarks, after sorted insertion and after each of ten random insertion orders; for the random orders, report the mean and the range. Compare the results with $\lg n$ and with the bounds in Section 1.1. Keep your benchmarks, raw results, and analysis scripts, with a list of their dependencies, in the repository, in folders of your choice other than `build/`, which Git ignores; state the compiler, operating system, and processor you used. Base your conclusions on relative comparisons rather than absolute times.

5 Literature Comparison

Compare your results with the article by Brown (2025), using the arXiv copy linked in the references, and cite its pages, tables, and figures. Consider only the AVL tree and the bottom-up red–black tree, and the times to insert and to delete 2^{18} keys, each in random and in sorted order, which Tables 1 and 2 of the article report relative to the fastest tree. For deletion, compare your AVL tree with the article's version of the deletion algorithm you implemented, and your red–black tree with customary deletion.

In your report, cover the following:

- **Conditions:** State how the article obtained these times, including the key type, node layout, repetitions, compiler options, and hardware.
- **Matched experiment:** Measure the same four times for both of your trees (Section 4.3), matching the article's conditions as closely as your setup allows.
- **Comparison:** For each of the four times, state which tree, if either, is faster in the article and in your results, and compare the ratios of AVL to red–black times. Also compare your sorted-to-random speed ratios with Table 4 of the article, which covers insertion and customary deletion.
- **Explanation:** Pick the ratio that differs from the article's by the largest factor, such as a case where the other tree is faster. Propose a cause, such as a difference in hardware, compiler, node layout, or workload, and test it with an additional experiment where possible.

6 Report

Write one report per team in the Word template `report/report_template.docx` from the starter code, keep its section headings and the wording of its declaration, and submit it as described in Section 8. The report may contain at most 2,500 words, a hard limit that excludes the title page and the reference list; text in tables, captions, and footnotes counts. Count the words with Word's Word Count dialog, with "Include textboxes, footnotes and endnotes" selected, and state the count at the end of the report. Present every measured result in a table or a plot, with at least one of each in the report. Do not add appendices. Cite every source that substantially influences your work.

Section	Content	Words
Title page	Team number, member names, the URL of your repository, and the declaration of proper AI use, signed by every team member	Not counted
Implementation	Design decisions for both trees, and your AVL deletion algorithm, identified as customary or alternative, with an argument that it preserves the AVL property in $O(\lg n)$ time	400
Testing	How your tests (Section 4.2 of the assignment brief) target likely defects, and the defects you found and fixed	250
Experiments	Method, results, and interpretation of the experiments in Section 4.3 of the assignment brief	700
Literature Comparison	The points listed in Section 5 of the assignment brief	450
AI Reflection	The points listed in Section 7.4 of the assignment brief	500
Conclusions	When and why one tree outperforms the other, based on your evidence	100
Contributions	What each team member contributed	100
References	All cited sources, in one consistent citation style	Not counted

The word counts per section are suggestions: a section may be longer or shorter, as long as the whole report stays within 2,500 words. Number and caption every table and figure, refer to each in the text, and refer to your code by file and function name.

7 Rules on AI Use

These rules apply to every team member and to all project work.

7.1 Permitted Use

- **Chat interfaces only:** Use one or more AI chat tools in a web browser, not their desktop or mobile apps. Do not use AI agents or autonomous modes, do not let AI access or edit your files, and do not ask it to run code or experiments; turn off code execution where the tool allows. Web search within the chat is permitted.
- **No AI in your editors:** Disable AI chat, completion, and writing tools, such as GitHub Copilot, IntelliCode, and Copilot in Word; IntelliSense and spelling and grammar checkers are permitted.
- **No file transfers:** Do not upload files to AI or download files it generates. Save your conversations for the logs yourselves, by export, browser extension, or copy and paste, never by asking AI to write them out (Section 7.3). Paste at most three paragraphs of prose or 100 lines of code per prompt.
- **No complete operations:** You may adopt short snippets that address a specific defect or design question, but never AI code for a whole operation, such as insertion, deletion, or rebalancing, even across prompts.
- **Your reasoning first:** Begin every request about your code, tests, experiments, or report with your own approach, draft, or diagnosis, and verify every suggestion before using it.

7.2 Reading the Article

Read the article's methods, results, and limitations yourself rather than an AI summary, and do not use deep-research modes that search and compile reports autonomously.

7.3 AI Interaction Logs

Log every project-related prompt and response of every team member verbatim, including unsuccessful exchanges and report-preparation discussions; undocumented side conversations with AI are not permitted. Store each conversation as a separate Markdown file in `ai_logs/`, named after its start date, the tool, and a short topic, for example `2026-10-02_chatgpt_rotation-bug.md`. Begin each file with a share link created after the conversation ends, and keep the link working until grades are released. Any layout, including plain text, is acceptable if it keeps the text unchanged and distinguishes your prompts from the AI's responses. Test your export method before starting substantial work; a summary or reconstruction written by AI is not a log.

7.4 AI Reflection

In the AI Reflection section of your report (Section 6), explain which tools you used and how you framed your requests, which suggestions helped and how you verified them, and which responses you amended or rejected and why. Support each claim with a reference to the log file and line numbers, and do not invent errors or rejected suggestions to appear more critical.

8 Deliverables and Submission

Submit the following by Friday, 23 October 2026, 23:59:

Where	What
GitHub	Your repository (Section 3), with your trees, tests, benchmarks, analysis scripts, and AI logs on the default branch; work on other branches is not assessed
Gradescope	Your repository at the assessed commit (Section 3), submitted to the assignment "Project 1: Comparing Red–Black and AVL Trees" as described below
xSITe	Your report as `team-<n>_report.docx`, uploaded by one team member; the final upload counts

After your final push, one team member submits to Gradescope in either of two ways and adds the others to the submission:

- **GitHub:** Choose your repository and its default branch.

- **Upload:** With the default branch checked out, run `git archive -o submission.zip HEAD` in your repository folder, and upload `submission.zip`.

Before the deadline, clone your repository into a new folder and check that it builds with `make` and contains everything listed above. Keep the repository private and accessible to the instructors, with its history intact, until grades are released. The time of your final xSITe upload decides whether the whole submission, code included, is late under the policy in the course information sheet. A push after that upload is not assessed unless you upload the report again.

9 Assessment

Project 1 contributes 12.5% of your course grade and is marked out of 100 points; all team members receive the same mark. Instructors may lower the mark of a member whose contribution is negligible in the Contributions section, the Git history, and the AI logs; raise problems in your team with an instructor early.

Component	What is assessed	Points
Correctness	Share of instructor tests passed (5) and share of other teams' valid tests passed (10)	15
Testing	Testing section of the report (5), and the number of other teams' trees that fail at least one of your valid tests, relative to the most effective team (5)	10
Implementation	Implementation section of the report, including the justification of your AVL deletion algorithm, and the clarity of your code	10
Experiments	Experiments and Conclusions sections of the report, and the benchmarks and scripts in your repository	20
Literature comparison	Literature Comparison section of the report	20
AI use	AI Reflection section of the report, checked against your AI logs	25
Total		**100**

The following deductions apply to the total:

Issue	Deduction
`tree` takes longer than one minute on a command file of the instructor tests (Section 2.3)	20 points
The instructors cannot access your repository when grading begins (Section 3)	10 points
The final Gradescope submission does not match the assessed commit (Section 3)	5 points
The report exceeds 2,500 words or does not state its word count (Section 6)	5 points

! Academic Integrity

Fabricating data or citations, falsifying interaction logs, or deliberately misrepresenting AI assistance constitutes academic misconduct and may lead to disciplinary proceedings.

References

Brown, R.A. (2025) 'Comparative performance of the AVL tree and three variants of the red–black tree'. arXiv:2406.05162v21. Available at: https://arxiv.org/abs/2406.05162v21.

Cormen, T.H. *et al.* (2022) *Introduction to algorithms*. 4th edn. MIT Press.